Max

Response provided by Anthropic

🚀

Hier ist der komplette, produktionsreife Code für alle 4 Features auf einmal. Ich erkläre die Architektur-Entscheidungen **vorausschauend**, bevor wir bauen.

🧠

```
text
```

```
┌──────────────────────────────────────────────┐
│                  RENDERER (UI)               │
│  Button klick → window.electronAPI.openFile()│
│       ↕ contextBridge (sicher, kein remote!) │
│                  PRELOAD.JS                  │
│       ↕ ipcRenderer.invoke() → Promise       │
│                  MAIN PROCESS                │
│  ipcMain.handle() → dialog.showOpenDialog()  │
└──────────────────────────────────────────────┘
```

> ⚠️ *Wichtig 2024/2025: 4 Du kannst Electron-Module nicht direkt im Renderer `require`n. Stattdessen musst du den Dialog über `contextBridge.exposeInMainWorld()` und den Preloader dem Renderer-Code zugänglich machen.*

> ⚠️ *Kritischer Bug in Electron 37: 1 `dialog.showOpenDialog()` mit `properties: ['openDirectory']` öffnet in Electron 37.x einen Datei-Picker statt einen Ordner-Picker — wir bleiben daher bei `openFile` für `.exe` und `.gguf`.*

> ✅ *Modernes Pattern: 11 Die offizielle Methode ist `ipcRenderer.invoke()` kombiniert mit `ipcMain.handle()`. Das funktioniert wie ein Request/Response-Call über Prozesse: Der Renderer sendet eine Anfrage, der Main-Prozess bearbeitet sie, das Ergebnis kommt als Promise zurück.*

📁

{} text

```
hermes-launcher/
├── main.js              ← Electron Main Process
├── preload.js           ← Secure Bridge
├── renderer/
│   ├── index.html       ← Haupt-UI
│   ├── setup.html       ← Setup-Assistent
│   ├── app.js           ← UI Logic
│   └── styles.css       ← Dark Theme
├── backend/
│   └── server.js        ← Express + llama-server Wrapper
└── package.json
```

---

1 main.js

{} JavaScript

```javascript
const { app, BrowserWindow, ipcMain, dialog, shell } = require('electron')
const path = require('path')
const { spawn } = require('child_process')
const Store = require('electron-store') // npm install electron-store

const store = new Store()
let mainWindow
let llamaProcess = null
let logBuffer = []

// ─────────────────────────────────────────
// Fenster erstellen
// ─────────────────────────────────────────
function createWindow() {
  mainWindow = new BrowserWindow({
    width: 1100,
    height: 750,
    minWidth: 800,
    minHeight: 600,
    backgroundColor: '#0f0f0f',
    titleBarStyle: 'hidden',           // Modernes look
    trafficLightPosition: { x: 16, y: 16 },
    webPreferences: {
      preload: path.join(__dirname, 'preload.js'),
      contextIsolation: true,          // ✅ PFLICHT
      nodeIntegration: false,          // ✅ Sicherheit
      sandbox: false
    }
  })
  mainWindow.loadFile('renderer/index.html')
}
```

```
app.whenReady().then(createWindow)

app.on('window-all-closed', () => {
  if (llamaProcess) llamaProcess.kill()
  if (process.platform !== 'darwin') app.quit()
})

// ──────────────────────────────────────────
// IPC: llama-server.exe auswählen
// ──────────────────────────────────────────
ipcMain.handle('select-llama-exe', async () => {
  const result = await dialog.showOpenDialog(mainWindow, {
    title: 'llama-server.exe auswählen',
    buttonLabel: 'Auswählen',
    defaultPath: store.get('llamaExePath', 'C:\\'),
    filters: [
      { name: 'Executables', extensions: ['exe'] },
      { name: 'Alle Dateien',  extensions: ['*'] }
    ],
    properties: ['openFile']
  })

  if (!result.canceled && result.filePaths.length > 0) {
    const exePath = result.filePaths[0]
    store.set('llamaExePath', exePath)   // Pfad persistent speichern
    return { success: true, path: exePath }
  }
  return { success: false }
})

// ──────────────────────────────────────────
// IPC: GGUF-Modell auswählen
// ──────────────────────────────────────────
ipcMain.handle('select-gguf-model', async () => {
  const result = await dialog.showOpenDialog(mainWindow, {
    title: 'GGUF-Modell auswählen',
    buttonLabel: 'Modell laden',
    defaultPath: store.get('ggufModelPath', 'C:\\'),
    filters: [
      { name: 'GGUF Modelle', extensions: ['gguf'] },
      { name: 'Alle Dateien', extensions: ['*']    }
    ],
    properties: ['openFile']
  })

  if (!result.canceled && result.filePaths.length > 0) {
    const modelPath = result.filePaths[0]
    store.set('ggufModelPath', modelPath)
    return { success: true, path: modelPath }
  }
  return { success: false }
})

// ──────────────────────────────────────────
// IPC: Gespeicherte Pfade laden
// ──────────────────────────────────────────
ipcMain.handle('get-saved-paths', () => {
```

```
  return {
    llamaExePath:  store.get('llamaExePath',  ''),
    ggufModelPath: store.get('ggufModelPath', ''),
    port:          store.get('port',          8080),
    ngl:           store.get('ngl',           35),
    ctxSize:       store.get('ctxSize',       4096)
  }
})

ipcMain.handle('save-settings', (_, settings) => {
  Object.entries(settings).forEach(([k, v]) => store.set(k, v))
  return { success: true }
})

// ──────────────────────────────────────────
// IPC: llama-server starten
// ──────────────────────────────────────────
ipcMain.handle('start-llama', async (_, config) => {
  if (llamaProcess) {
    return { success: false, error: 'Server läuft bereits!' }
  }

  const { exePath, modelPath, port, ngl, ctxSize } = config

  if (!exePath || !modelPath) {
    return { success: false, error: 'Exe und Modell müssen ausgewählt sein.' }
  }

  const args = [
    '-m',    modelPath,
    '--port', String(port  || 8080),
    '-ngl',   String(ngl   || 35),
    '-c',     String(ctxSize || 4096),
    '--host', '127.0.0.1'
  ]

  try {
    llamaProcess = spawn(exePath, args, {
      env: { ...process.env },
      windowsHide: true           // Kein schwarzes CMD-Fenster
    })

    // ─── STDOUT → Log-Buffer + Renderer ───
    llamaProcess.stdout.on('data', (data) => {
      const lines = data.toString().split('\n').filter(l => l.trim())
      lines.forEach(line => {
        logBuffer.push({ ts: Date.now(), level: 'info', msg: line })
        if (logBuffer.length > 2000) logBuffer.shift()  // Max 2000 Einträge
        mainWindow?.webContents.send('log-line', { level: 'info', msg: line })
      })
    })

    // ─── STDERR → Log-Buffer + Renderer ───
    llamaProcess.stderr.on('data', (data) => {
      const lines = data.toString().split('\n').filter(l => l.trim())
      lines.forEach(line => {
        const level = line.toLowerCase().includes('error') ? 'error' : 'warn'
```

```
        logBuffer.push({ ts: Date.now(), level, msg: line })
        if (logBuffer.length > 2000) logBuffer.shift()
        mainWindow?.webContents.send('log-line', { level, msg: line })
      })
    })

    llamaProcess.on('close', (code) => {
      const msg = `llama-server beendet (Exit-Code: ${code})`
      logBuffer.push({ ts: Date.now(), level: 'warn', msg })
      mainWindow?.webContents.send('log-line',      { level: 'warn', msg })
      mainWindow?.webContents.send('server-stopped', { code })
      llamaProcess = null
    })

    llamaProcess.on('error', (err) => {
      mainWindow?.webContents.send('log-line', {
        level: 'error',
        msg: `Spawn-Fehler: ${err.message}`
      })
      llamaProcess = null
    })

    return { success: true }

  } catch (err) {
    return { success: false, error: err.message }
  }
})

// ─────────────────────────────────────────
// IPC: llama-server stoppen
// ─────────────────────────────────────────
ipcMain.handle('stop-llama', () => {
  if (llamaProcess) {
    llamaProcess.kill('SIGTERM')
    return { success: true }
  }
  return { success: false, error: 'Kein Prozess aktiv.' }
})

// ─────────────────────────────────────────
// IPC: Log-History abrufen (beim Öffnen)
// ─────────────────────────────────────────
ipcMain.handle('get-log-history', () => logBuffer)

// ─────────────────────────────────────────
// IPC: Im Explorer öffnen
// ─────────────────────────────────────────
ipcMain.handle('open-in-explorer', (_, filePath) => {
  shell.showItemInFolder(filePath)
  return { success: true }
})
```

2 preload.js

JavaScript

```
const { contextBridge, ipcRenderer } = require('electron')

// ─────────────────────────────────────────────────
// Alle erlaubten Kanäle whitelisten – Sicherheit!
// ─────────────────────────────────────────────────
contextBridge.exposeInMainWorld('electronAPI', {

  // ── Dateiauswahl ──
  selectLlamaExe:   () => ipcRenderer.invoke('select-llama-exe'),
  selectGgufModel:  () => ipcRenderer.invoke('select-gguf-model'),

  // ── Einstellungen ──
  getSavedPaths:    () => ipcRenderer.invoke('get-saved-paths'),
  saveSettings:  (s) => ipcRenderer.invoke('save-settings', s),

  // ── Server-Steuerung ──
  startLlama:    (c) => ipcRenderer.invoke('start-llama', c),
  stopLlama:        () => ipcRenderer.invoke('stop-llama'),

  // ── Log-System ──
  getLogHistory:    () => ipcRenderer.invoke('get-log-history'),
  onLogLine:     (cb) => ipcRenderer.on('log-line',      (_, d) => cb(d)),
  onServerStop:  (cb) => ipcRenderer.on('server-stopped',(_, d) => cb(d)),

  // ── Explorer ──
  openInExplorer:(p)  => ipcRenderer.invoke('open-in-explorer', p),

  // ── Listener aufräumen ──
  removeAllListeners: (channel) => ipcRenderer.removeAllListeners(channel)
})
```

---

3 renderer/index.html

HTML

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="Content-Security-Policy"
        content="default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'">
  <title>Hermes Launcher</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
```

```
<!-- ═══ HEADER ═══════════════════════════════════════ -->
<header class="app-header">
  <div class="logo">🪁 Hermes Launcher</div>
  <div class="header-status">
    <span id="status-dot" class="dot dot-off"></span>
    <span id="status-text">Gestoppt</span>
  </div>
</header>

<!-- ═══ MAIN GRID ═══════════════════════════════════ -->
<main class="grid">

  <!-- LINKE SPALTE: Setup ──────────────────────── -->
  <section class="card setup-card">
    <h2>⚙️ Setup</h2>

    <!-- llama-server.exe -->
    <div class="field-group">
      <label>llama-server.exe</label>
      <div class="path-row">
        <input id="exe-path"   type="text" readonly placeholder="Noch nicht ausgewählt…">
        <button id="btn-exe"   class="btn btn-pick">📂 Auswählen</button>
        <button id="btn-exe-open" class="btn btn-icon" title="Im Explorer öffnen">🔍</button>
      </div>
    </div>

    <!-- GGUF-Modell -->
    <div class="field-group">
      <label>GGUF-Modell</label>
      <div class="path-row">
        <input id="model-path" type="text" readonly placeholder="Noch nicht ausgewählt…">
        <button id="btn-model"      class="btn btn-pick">📂 Auswählen</button>
        <button id="btn-model-open" class="btn btn-icon" title="Im Explorer öffnen">🔍</button>
      </div>
    </div>

    <!-- Erweiterte Parameter -->
    <details class="advanced-section">
      <summary>🔧 Erweiterte Parameter</summary>
      <div class="params-grid">
        <div class="param-field">
          <label>Port</label>
          <input id="param-port"    type="number" value="8080" min="1024" max="65535">
        </div>
        <div class="param-field">
          <label>GPU-Layer (-ngl)</label>
          <input id="param-ngl"     type="number" value="35" min="0" max="200">
        </div>
        <div class="param-field">
          <label>Kontext-Größe (-c)</label>
          <input id="param-ctx"     type="number" value="4096" min="512" max="131072">
        </div>
        <div class="param-field">
          <label>Threads (-t)</label>
          <input id="param-threads" type="number" value="8" min="1" max="64">
        </div>
```

```
        </div>
      </details>

      <!-- Start/Stop Buttons -->
      <div class="action-row">
        <button id="btn-start" class="btn btn-start">▶ Server starten</button>
        <button id="btn-stop"  class="btn btn-stop" disabled>■ Stoppen</button>
        <button id="btn-wizard" class="btn btn-wizard">🧙 Setup-Assistent</button>
      </div>

      <!-- Server-Info -->
      <div id="server-info" class="server-info hidden">
        <span>🌐 API: <a id="api-link" href="#" target="_blank">http://127.0.0.1:8080</a></span>
        <span id="uptime-counter">⏱ 0s</span>
      </div>
    </section>

    <!-- RECHTE SPALTE: Log-Viewer ──────────────────── -->
    <section class="card log-card">
      <div class="log-header">
        <h2>📋 Server-Log</h2>
        <div class="log-controls">
          <!-- Filter -->
          <label class="filter-toggle">
            <input type="checkbox" id="filter-errors" > Nur Fehler
          </label>
          <label class="filter-toggle">
            <input type="checkbox" id="filter-autoscroll" checked> Auto-Scroll
          </label>
          <!-- Suche -->
          <input id="log-search" type="text" placeholder="🔍 Log durchsuchen…" class="log-search">
          <!-- Buttons -->
          <button id="btn-clear-log"  class="btn btn-sm">🗑 Leeren</button>
          <button id="btn-export-log" class="btn btn-sm">💾 Exportieren</button>
        </div>
      </div>

      <!-- Log-Ausgabe -->
      <div id="log-viewer" class="log-viewer">
        <div class="log-placeholder">Starte den Server, um Logs zu sehen…</div>
      </div>

      <!-- Log-Statusbar -->
      <div class="log-statusbar">
        <span id="log-count">0 Einträge</span>
        <span id="log-filter-hint"></span>
      </div>
    </section>

  </main>

  <!-- ═══ SETUP-ASSISTENT MODAL ═══════════════════ -->
  <div id="wizard-overlay" class="overlay hidden">
    <div class="wizard-modal">
      <button class="close-btn" id="close-wizard">✕</button>
      <div id="wizard-content">
        <!-- Dynamisch befüllt durch app.js -->
```

```
      </div>
    </div>
  </div>

  <script src="app.js"></script>
</body>
</html>
```

---

4 renderer/app.js

{} JavaScript

```
// ═══════════════════════════════════════════════════
//  Hermes Launcher — Renderer Logic
// ═══════════════════════════════════════════════════

const api = window.electronAPI

// ── State ──────────────────────────────────────────
let state = {
  exePath:    '',
  modelPath:  '',
  port:       8080,
  ngl:        35,
  ctxSize:    4096,
  threads:    8,
  running:    false,
  startTime:  null,
  logEntries: [],
  uptimeTimer: null
}

// ── DOM-Refs ───────────────────────────────────────
const dom = {
  exePath:       document.getElementById('exe-path'),
  modelPath:     document.getElementById('model-path'),
  btnExe:        document.getElementById('btn-exe'),
  btnExeOpen:    document.getElementById('btn-exe-open'),
  btnModel:      document.getElementById('btn-model'),
  btnModelOpen:  document.getElementById('btn-model-open'),
  btnStart:      document.getElementById('btn-start'),
  btnStop:       document.getElementById('btn-stop'),
  btnWizard:     document.getElementById('btn-wizard'),
  statusDot:     document.getElementById('status-dot'),
  statusText:    document.getElementById('status-text'),
  logViewer:     document.getElementById('log-viewer'),
  logCount:      document.getElementById('log-count'),
  logSearch:     document.getElementById('log-search'),
  filterErrors:  document.getElementById('filter-errors'),
  filterScroll:  document.getElementById('filter-autoscroll'),
  btnClearLog:   document.getElementById('btn-clear-log'),
```

```
  btnExportLog:  document.getElementById('btn-export-log'),
  serverInfo:    document.getElementById('server-info'),
  apiLink:       document.getElementById('api-link'),
  uptimeCounter: document.getElementById('uptime-counter'),
  wizardOverlay: document.getElementById('wizard-overlay'),
  wizardContent: document.getElementById('wizard-content'),
  paramPort:     document.getElementById('param-port'),
  paramNgl:      document.getElementById('param-ngl'),
  paramCtx:      document.getElementById('param-ctx'),
  paramThreads:  document.getElementById('param-threads'),
}

// ═══════════════════════════════════════════════════
//  INIT
// ═══════════════════════════════════════════════════
async function init() {
  // Gespeicherte Pfade laden
  const saved = await api.getSavedPaths()
  if (saved.llamaExePath)  setExePath(saved.llamaExePath)
  if (saved.ggufModelPath) setModelPath(saved.ggufModelPath)
  if (saved.port)    dom.paramPort.value    = saved.port
  if (saved.ngl)     dom.paramNgl.value     = saved.ngl
  if (saved.ctxSize) dom.paramCtx.value     = saved.ctxSize

  // Log-History laden (falls App neu geöffnet)
  const history = await api.getLogHistory()
  history.forEach(entry => appendLog(entry.level, entry.msg, false))

  // Live-Log-Listener
  api.onLogLine(({ level, msg }) => appendLog(level, msg, true))

  // Server-Stop-Listener
  api.onServerStop(({ code }) => setServerState(false))

  attachEventListeners()
  updateLogCount()
}

// ═══════════════════════════════════════════════════
//  EVENT LISTENER
// ═══════════════════════════════════════════════════
function attachEventListeners() {

  // ── Dateiauswahl ──────────────────────────────
  dom.btnExe.addEventListener('click', async () => {
    dom.btnExe.disabled = true
    dom.btnExe.textContent = '⏳ Wähle…'
    const result = await api.selectLlamaExe()
    dom.btnExe.disabled = false
    dom.btnExe.textContent = '📂 Auswählen'
    if (result.success) setExePath(result.path)
  })

  dom.btnModel.addEventListener('click', async () => {
    dom.btnModel.disabled = true
    dom.btnModel.textContent = '⏳ Wähle…'
    const result = await api.selectGgufModel()
```

```
    dom.btnModel.disabled = false
    dom.btnModel.textContent = '📂 Auswählen'
    if (result.success) setModelPath(result.path)
  })

  // ── Im Explorer öffnen ──────────────────────
  dom.btnExeOpen.addEventListener('click', () => {
    if (state.exePath) api.openInExplorer(state.exePath)
  })

  dom.btnModelOpen.addEventListener('click', () => {
    if (state.modelPath) api.openInExplorer(state.modelPath)
  })

  // ── Server Start / Stop ─────────────────────
  dom.btnStart.addEventListener('click', startServer)
  dom.btnStop.addEventListener('click',  stopServer)

  // ── Wizard ──────────────────────────────────
  dom.btnWizard.addEventListener('click', openWizard)
  document.getElementById('close-wizard')
          .addEventListener('click', closeWizard)
  dom.wizardOverlay.addEventListener('click', (e) => {
    if (e.target === dom.wizardOverlay) closeWizard()
  })

  // ── Log-Controls ────────────────────────────
  dom.logSearch.addEventListener('input',     renderLog)
  dom.filterErrors.addEventListener('change', renderLog)
  dom.btnClearLog.addEventListener('click', () => {
    state.logEntries = []
    renderLog()
  })
  dom.btnExportLog.addEventListener('click', exportLog)

  // ── ESC schließt Modal ──────────────────────
  document.addEventListener('keydown', (e) => {
    if (e.key === 'Escape') closeWizard()
  })
}

// ═══════════════════════════════════════════════
//  HELPER: Pfade setzen
// ═══════════════════════════════════════════════
function setExePath(p) {
  state.exePath = p
  dom.exePath.value = p
  dom.exePath.title = p
  dom.btnExeOpen.disabled = !p
  validateStartButton()
}

function setModelPath(p) {
  state.modelPath = p
  dom.modelPath.value = p
  dom.modelPath.title = p
  dom.btnModelOpen.disabled = !p
```

```
  validateStartButton()
}

function validateStartButton() {
  dom.btnStart.disabled = state.running || !state.exePath || !state.modelPath
}

// ═══════════════════════════════════════════════
//  SERVER START / STOP
// ═══════════════════════════════════════════════
async function startServer() {
  const config = {
    exePath:   state.exePath,
    modelPath: state.modelPath,
    port:      parseInt(dom.paramPort.value)    || 8080,
    ngl:       parseInt(dom.paramNgl.value)     || 35,
    ctxSize:   parseInt(dom.paramCtx.value)     || 4096,
    threads:   parseInt(dom.paramThreads.value) || 8
  }

  // Einstellungen speichern
  await api.saveSettings({
    port:    config.port,
    ngl:     config.ngl,
    ctxSize: config.ctxSize
  })

  appendLog('info', `🚀 Starte llama-server mit Modell: ${config.modelPath}`, true)

  const result = await api.startLlama(config)

  if (result.success) {
    setServerState(true, config.port)
  } else {
    appendLog('error', `❌ Fehler: ${result.error}`, true)
  }
}

async function stopServer() {
  const result = await api.stopLlama()
  if (!result.success) {
    appendLog('error', `❌ Stop-Fehler: ${result.error}`, true)
  }
}

function setServerState(running, port) {
  state.running = running

  if (running) {
    state.startTime = Date.now()
    dom.statusDot.className  = 'dot dot-on'
    dom.statusText.textContent = 'Läuft'
    dom.btnStart.disabled    = true
    dom.btnStop.disabled     = false
    dom.serverInfo.classList.remove('hidden')
    dom.apiLink.href = `http://127.0.0.1:${port || 8080}`
    dom.apiLink.textContent = `http://127.0.0.1:${port || 8080}`
```

```
    startUptimeCounter()
  } else {
    dom.statusDot.className  = 'dot dot-off'
    dom.statusText.textContent = 'Gestoppt'
    dom.btnStart.disabled    = !state.exePath || !state.modelPath
    dom.btnStop.disabled     = true
    dom.serverInfo.classList.add('hidden')
    stopUptimeCounter()
  }
}

function startUptimeCounter() {
  stopUptimeCounter()
  state.uptimeTimer = setInterval(() => {
    const s = Math.floor((Date.now() - state.startTime) / 1000)
    const h = Math.floor(s / 3600)
    const m = Math.floor((s % 3600) / 60)
    const sec = s % 60
    dom.uptimeCounter.textContent =
      `⏱ ${h > 0 ? h + 'h ' : ''}${m > 0 ? m + 'm ' : ''}${sec}s`
  }, 1000)
}

function stopUptimeCounter() {
  if (state.uptimeTimer) {
    clearInterval(state.uptimeTimer)
    state.uptimeTimer = null
  }
}

// ═══════════════════════════════════════════════
//  LOG-VIEWER
// ═══════════════════════════════════════════════
function appendLog(level, msg, doRender = true) {
  const entry = {
    id:    state.logEntries.length,
    ts:    new Date().toLocaleTimeString('de-DE'),
    level,
    msg:   msg.trim()
  }
  state.logEntries.push(entry)
  if (state.logEntries.length > 5000) state.logEntries.shift()
  if (doRender) renderLog()
}

function renderLog() {
  const query       = dom.logSearch.value.toLowerCase()
  const errorsOnly  = dom.filterErrors.checked
  const autoScroll  = dom.filterScroll.checked

  // Placeholder entfernen
  const placeholder = dom.logViewer.querySelector('.log-placeholder')
  if (placeholder) placeholder.remove()

  const filtered = state.logEntries.filter(e => {
    if (errorsOnly && e.level === 'info') return false
    if (query && !e.msg.toLowerCase().includes(query)) return false
```

```
    return true
  })

  // Efficient DOM-Update: nur neue Zeilen anhängen wenn kein Filter aktiv
  if (!query && !errorsOnly) {
    // Schnell: nur letzte fehlende Zeilen anhängen
    const existing = dom.logViewer.children.length
    const toAdd    = filtered.slice(existing)
    toAdd.forEach(e => dom.logViewer.appendChild(createLogLine(e)))
  } else {
    // Filter aktiv: komplett neu rendern
    dom.logViewer.innerHTML = ''
    filtered.forEach(e => dom.logViewer.appendChild(createLogLine(e)))
  }

  updateLogCount(filtered.length)

  if (autoScroll) {
    dom.logViewer.scrollTop = dom.logViewer.scrollHeight
  }
}

function createLogLine(entry) {
  const el = document.createElement('div')
  el.className = `log-line log-${entry.level}`

  const tsSpan  = document.createElement('span')
  tsSpan.className = 'log-ts'
  tsSpan.textContent = entry.ts

  const lvlSpan = document.createElement('span')
  lvlSpan.className = 'log-lvl'
  lvlSpan.textContent = entry.level.toUpperCase().padEnd(5)

  const msgSpan = document.createElement('span')
  msgSpan.className = 'log-msg'
  msgSpan.textContent = entry.msg

  // Suchbegriff highlighten
  const query = dom.logSearch.value
  if (query) {
    const re = new RegExp(`(${escapeRegex(query)})`, 'gi')
    msgSpan.innerHTML = entry.msg.replace(re, '<mark>$1</mark>')
  }

  el.appendChild(tsSpan)
  el.appendChild(lvlSpan)
  el.appendChild(msgSpan)
  return el
}

function updateLogCount(filtered) {
  const total = state.logEntries.length
  dom.logCount.textContent =
    filtered !== undefined && filtered !== total
      ? `${filtered} / ${total} Einträge`
      : `${total} Einträge`
```

```
}

function exportLog() {
  const text = state.logEntries
    .map(e => `[${e.ts}] [${e.level.toUpperCase()}] ${e.msg}`)
    .join('\n')
  const blob = new Blob([text], { type: 'text/plain' })
  const url  = URL.createObjectURL(blob)
  const a    = document.createElement('a')
  a.href     = url
  a.download = `hermes-log-${Date.now()}.txt`
  a.click()
  URL.revokeObjectURL(url)
}

function escapeRegex(str) {
  return str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')
}

// ═══════════════════════════════════════════════════
//  HERMES SETUP-ASSISTENT (Wizard)
// ═══════════════════════════════════════════════════
const WIZARD_STEPS = [
  {
    id: 'welcome',
    title: '👋 Willkommen beim Hermes Setup-Assistenten',
    content: `
      <p>Dieser Assistent führt dich in <strong>4 Schritten</strong> durch die Einrichtung.</p>
      <ul>
        <li>✅ llama-server.exe finden</li>
        <li>✅ GGUF-Modell auswählen</li>
        <li>✅ Parameter optimieren</li>
        <li>✅ Ersten Start durchführen</li>
      </ul>
      <p class="hint">💡 Tipp: Du brauchst llama.cpp –
        <a href="https://github.com/ggml-org/llama.cpp/releases" target="_blank">
          aktuelle Releases hier
        </a>
      </p>
    `,
    action: null
  },
  {
    id: 'exe',
    title: '📦 Schritt 1: llama-server.exe auswählen',
    content: `
      <p>Wähle die <code>llama-server.exe</code> aus deiner llama.cpp-Installation.</p>
      <p class="hint">📁 Normalerweise in: <code>llama.cpp\\build\\bin\\Release\\</code></p>
      <div id="wizard-exe-status" class="wizard-status"></div>
    `,
    action: async () => {
      const result = await api.selectLlamaExe()
      if (result.success) {
        setExePath(result.path)
        document.getElementById('wizard-exe-status').innerHTML =
          `<span class="ok">✅ Ausgewählt: ${result.path}</span>`
        return true
```

```
      }
      return false
    }
  },
  {
    id: 'model',
    title: '🧠 Schritt 2: GGUF-Modell auswählen',
    content: `
      <p>Wähle dein <code>.gguf</code>-Modell aus.</p>
      <p class="hint">
        💡 Empfohlene Modelle für den Start:<br>
        • <strong>Hermes-3-Llama-3.2-3B.Q4_K_M.gguf</strong> (sehr schnell, ~2GB)<br>
        • <strong>Mistral-7B-Instruct-v0.3.Q4_K_M.gguf</strong> (~4GB)<br>
        • <strong>Llama-3.1-8B-Instruct-Q4_K_M.gguf</strong> (~5GB)
      </p>
      <div id="wizard-model-status" class="wizard-status"></div>
    `,
    action: async () => {
      const result = await api.selectGgufModel()
      if (result.success) {
        setModelPath(result.path)
        document.getElementById('wizard-model-status').innerHTML =
          `<span class="ok">✅ Ausgewählt: ${result.path}</span>`
        return true
      }
      return false
    }
  },
  {
    id: 'params',
    title: '🔧 Schritt 3: Parameter einstellen',
    content: `
      <div class="wizard-params">
        <div class="param-block">
          <h4>GPU-Layer (-ngl)</h4>
          <p>Wie viele Schichten auf die GPU ausgelagert werden.</p>
          <div class="preset-row">
            <button class="preset-btn" onclick="setWizardNgl(0)">
              🖥️ CPU only<br><small>ngl=0</small>
            </button>
            <button class="preset-btn" onclick="setWizardNgl(20)">
              ⚡ Gemischt<br><small>ngl=20</small>
            </button>
            <button class="preset-btn active" onclick="setWizardNgl(35)">
              🚀 Standard<br><small>ngl=35</small>
            </button>
            <button class="preset-btn" onclick="setWizardNgl(99)">
              🔥 Volle GPU<br><small>ngl=99</small>
            </button>
          </div>
          <input id="wizard-ngl" type="number" value="35" min="0" max="200">
        </div>

        <div class="param-block">
          <h4>Kontext-Größe (-c)</h4>
          <p>Token-Fenster für Konversationen.</p>
          <div class="preset-row">
```

```
            <button class="preset-btn" onclick="setWizardCtx(2048)">
              💾 2K<br><small>sparsam</small>
            </button>
            <button class="preset-btn active" onclick="setWizardCtx(4096)">
              📋 4K<br><small>standard</small>
            </button>
            <button class="preset-btn" onclick="setWizardCtx(8192)">
              📚 8K<br><small>groß</small>
            </button>
            <button class="preset-btn" onclick="setWizardCtx(32768)">
              🐋 32K<br><small>max</small>
            </button>
          </div>
          <input id="wizard-ctx" type="number" value="4096" min="512">
        </div>
      </div>
    `,
    action: async () => {
      const ngl = parseInt(document.getElementById('wizard-ngl')?.value || '35')
      const ctx = parseInt(document.getElementById('wizard-ctx')?.value || '4096')
      dom.paramNgl.value = ngl
      dom.paramCtx.value = ctx
      await api.saveSettings({ ngl, ctxSize: ctx })
      return true
    }
  },
  {
    id: 'done',
    title: '🎉 Setup abgeschlossen!',
    content: `
      <div class="wizard-done">
        <div class="done-icon">🚀</div>
        <p>Alles ist bereit! Klicke unten auf <strong>Server starten</strong>.</p>
        <div class="config-summary" id="wizard-summary"></div>
        <p class="hint">
          🍼 Nach dem Start ist die API verfügbar unter:<br>
          <code>http://127.0.0.1:8080/v1</code>
        </p>
      </div>
    `,
    action: null,
    onShow: () => {
      const summary = document.getElementById('wizard-summary')
      if (summary) {
        summary.innerHTML = `
          <div class="summary-line">🖥️ <b>Exe:</b> <span>${state.exePath || '–'}</span></div>
          <div class="summary-line">🧠 <b>Modell:</b> <span>${state.modelPath || '–'}</span></div>
          <div class="summary-line">⚡ <b>GPU-Layer:</b> <span>${dom.paramNgl.value}</span></div>
          <div class="summary-line">📋 <b>Kontext:</b> <span>${dom.paramCtx.value} Token</span></div>
          <div class="summary-line">🌐 <b>Port:</b> <span>${dom.paramPort.value}</span></div>
        `
      }
    }
  }
]

let wizardStep = 0
```

```
function openWizard() {
  wizardStep = 0
  renderWizardStep()
  dom.wizardOverlay.classList.remove('hidden')
}

function closeWizard() {
  dom.wizardOverlay.classList.add('hidden')
}

function renderWizardStep() {
  const step = WIZARD_STEPS[wizardStep]
  const total = WIZARD_STEPS.length

  dom.wizardContent.innerHTML = `
    <div class="wizard-progress">
      ${WIZARD_STEPS.map((s, i) => `
        <div class="progress-step ${i === wizardStep ? 'active' : i < wizardStep ? 'done' : ''}">
          <div class="progress-dot">${i < wizardStep ? '✓' : i + 1}</div>
          <div class="progress-label">${s.id}</div>
        </div>
      `).join('<div class="progress-line"></div>')}
    </div>

    <div class="wizard-body">
      <h3>${step.title}</h3>
      <div class="wizard-text">${step.content}</div>
    </div>

    <div class="wizard-footer">
      <button class="btn"
              onclick="wizardBack()"
              ${wizardStep === 0 ? 'disabled' : ''}>
        ← Zurück
      </button>

      ${step.action ? `
        <button class="btn btn-pick" onclick="wizardAction()">
          📂 Datei auswählen
        </button>
      ` : ''}

      ${wizardStep < total - 1 ? `
        <button class="btn btn-start" onclick="wizardNext()">
          Weiter →
        </button>
      ` : `
        <button class="btn btn-start" onclick="wizardFinish()">
          🚀 Starten & Schließen
        </button>
      `}
    </div>
  `

  if (step.onShow) step.onShow()
}
```

```
async function wizardAction() {
  const step = WIZARD_STEPS[wizardStep]
  if (step.action) await step.action()
}

function wizardNext() {
  if (wizardStep < WIZARD_STEPS.length - 1) {
    wizardStep++
    renderWizardStep()
  }
}

function wizardBack() {
  if (wizardStep > 0) {
    wizardStep--
    renderWizardStep()
  }
}

function wizardFinish() {
  closeWizard()
  if (state.exePath && state.modelPath) {
    startServer()
  }
}

// Globale Hilfsfunktionen für den Wizard (onclick in HTML)
window.wizardNext   = wizardNext
window.wizardBack   = wizardBack
window.wizardAction = wizardAction
window.wizardFinish = wizardFinish

window.setWizardNgl = (val) => {
  const input = document.getElementById('wizard-ngl')
  if (input) input.value = val
  document.querySelectorAll('.preset-btn').forEach(b => {
    b.classList.toggle('active', b.textContent.includes(`ngl=${val}`))
  })
}

window.setWizardCtx = (val) => {
  const input = document.getElementById('wizard-ctx')
  if (input) input.value = val
}

// ═══════════════════════════════════════════════════
//  START
// ═══════════════════════════════════════════════════
init()
```

---

5 renderer/styles.css

{} CSS

```css
/* ═══════════════════════════════════════════════
   Hermes Launcher — Dark Theme CSS
═══════════════════════════════════════════════ */

:root {
  --bg:        #0f0f0f;
  --surface:   #1a1a1a;
  --surface2:  #242424;
  --border:    #333;
  --accent:    #7c3aed;
  --accent-h:  #6d28d9;
  --green:     #22c55e;
  --red:       #ef4444;
  --yellow:    #eab308;
  --text:      #e5e5e5;
  --text-dim:  #888;
  --font-mono: 'Cascadia Code', 'Fira Code', 'Consolas', monospace;
}

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  background: var(--bg);
  color: var(--text);
  height: 100vh;
  display: flex;
  flex-direction: column;
  overflow: hidden;
  user-select: none;
}

/* ── Header ──────────────────────────────────── */
.app-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px 20px;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  -webkit-app-region: drag;
  min-height: 48px;
}

.logo {
  font-size: 1.1rem;
  font-weight: 700;
  color: var(--accent);
  letter-spacing: 0.5px;
}

.header-status {
  display: flex;
  align-items: center;
```

```
  gap: 8px;
  font-size: 0.85rem;
  -webkit-app-region: no-drag;
}

/* Status-Dot */
.dot {
  width: 10px; height: 10px;
  border-radius: 50%;
  display: inline-block;
}
.dot-off { background: var(--text-dim); }
.dot-on  {
  background: var(--green);
  box-shadow: 0 0 6px var(--green);
  animation: pulse 2s infinite;
}

@keyframes pulse {
  0%, 100% { box-shadow: 0 0 4px var(--green); }
  50%      { box-shadow: 0 0 12px var(--green); }
}

/* ── Main Grid ──────────────────────────────── */
main.grid {
  display: grid;
  grid-template-columns: 420px 1fr;
  gap: 0;
  flex: 1;
  overflow: hidden;
}

/* ── Cards ──────────────────────────────────── */
.card {
  padding: 20px;
  overflow-y: auto;
  border-right: 1px solid var(--border);
}

.card h2 {
  font-size: 1rem;
  font-weight: 600;
  margin-bottom: 16px;
  color: var(--text-dim);
  text-transform: uppercase;
  letter-spacing: 1px;
}

/* ── Setup Card ─────────────────────────────── */
.setup-card { background: var(--surface); }

.field-group {
  margin-bottom: 14px;
}

.field-group label {
  display: block;
```

```
  font-size: 0.8rem;
  color: var(--text-dim);
  margin-bottom: 6px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.path-row {
  display: flex;
  gap: 6px;
  align-items: center;
}

.path-row input {
  flex: 1;
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 8px 10px;
  border-radius: 6px;
  font-size: 0.82rem;
  font-family: var(--font-mono);
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* ── Buttons ──────────────────────────────────── */
.btn {
  padding: 8px 14px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 0.82rem;
  font-weight: 500;
  transition: all 0.15s ease;
  white-space: nowrap;
}

.btn:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}

.btn-pick   { background: var(--surface2); color: var(--text); border: 1px solid var(--border); }
.btn-pick:hover:not(:disabled)  { background: var(--border); }

.btn-icon   { background: transparent; color: var(--text-dim); padding: 8px; }
.btn-icon:hover:not(:disabled)  { color: var(--text); }

.btn-start  { background: var(--accent); color: white; }
.btn-start:hover:not(:disabled) { background: var(--accent-h); }

.btn-stop   { background: var(--red); color: white; }
.btn-stop:hover:not(:disabled)  { opacity: 0.85; }
```

```css
.btn-wizard { background: var(--surface2); color: var(--text); border: 1px solid var(--accent); }
.btn-wizard:hover { background: var(--accent); color: white; }

.btn-sm { padding: 5px 10px; font-size: 0.78rem; background: var(--surface2);
          color: var(--text-dim); border: 1px solid var(--border); }
.btn-sm:hover { color: var(--text); }

/* ── Advanced / Params ──────────────────────── */
.advanced-section {
  margin: 14px 0;
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
}

.advanced-section summary {
  padding: 10px 14px;
  cursor: pointer;
  font-size: 0.85rem;
  color: var(--text-dim);
  list-style: none;
}
.advanced-section summary:hover { color: var(--text); }

.params-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  padding: 14px;
}

.param-field label {
  font-size: 0.75rem;
  color: var(--text-dim);
  display: block;
  margin-bottom: 4px;
}

.param-field input {
  width: 100%;
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 6px 8px;
  border-radius: 5px;
  font-size: 0.85rem;
}

/* ── Action Row ─────────────────────────────── */
.action-row {
  display: flex;
  gap: 8px;
  margin-top: 16px;
  flex-wrap: wrap;
}
```

```css
/* ── Server Info ──────────────────────────────── */
.server-info {
  margin-top: 14px;
  padding: 10px 14px;
  background: var(--surface2);
  border: 1px solid var(--green);
  border-radius: 6px;
  display: flex;
  gap: 16px;
  align-items: center;
  font-size: 0.82rem;
}

.server-info a { color: var(--accent); text-decoration: none; }
.server-info a:hover { text-decoration: underline; }

.hidden { display: none !important; }

/* ── Log Card ─────────────────────────────────── */
.log-card {
  display: flex;
  flex-direction: column;
  padding: 0;
  background: var(--bg);
  border-right: none;
  overflow: hidden;
}

.log-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px 16px;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  flex-wrap: wrap;
  gap: 8px;
}

.log-header h2 { margin: 0; }

.log-controls {
  display: flex;
  align-items: center;
  gap: 8px;
  flex-wrap: wrap;
}

.filter-toggle {
  display: flex;
  align-items: center;
  gap: 4px;
  font-size: 0.8rem;
  color: var(--text-dim);
  cursor: pointer;
}
```

```css
.log-search {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 5px 10px;
  border-radius: 6px;
  font-size: 0.8rem;
  width: 180px;
}

/* ── Log Viewer ──────────────────────────────── */
.log-viewer {
  flex: 1;
  overflow-y: auto;
  padding: 8px 0;
  font-family: var(--font-mono);
  font-size: 0.78rem;
  line-height: 1.5;
}

.log-placeholder {
  padding: 40px;
  text-align: center;
  color: var(--text-dim);
  font-family: -apple-system, sans-serif;
}

.log-line {
  display: flex;
  gap: 8px;
  padding: 2px 14px;
  border-bottom: 1px solid transparent;
}

.log-line:hover { background: var(--surface2); }

.log-info  { color: var(--text); }
.log-warn  { color: var(--yellow); }
.log-error { color: var(--red); background: rgba(239, 68, 68, 0.05); }

.log-ts  { color: var(--text-dim); min-width: 70px; }
.log-lvl { min-width: 50px; opacity: 0.7; }
.log-msg { flex: 1; word-break: break-all; }

.log-viewer mark {
  background: rgba(124, 58, 237, 0.35);
  color: white;
  border-radius: 2px;
  padding: 0 2px;
}

/* ── Scrollbar ──────────────────────────────── */
.log-viewer::-webkit-scrollbar { width: 6px; }
.log-viewer::-webkit-scrollbar-track { background: var(--bg); }
.log-viewer::-webkit-scrollbar-thumb { background: var(--border); border-radius: 3px; }

.log-statusbar {
```

```
  padding: 6px 14px;
  background: var(--surface);
  border-top: 1px solid var(--border);
  font-size: 0.75rem;
  color: var(--text-dim);
  display: flex;
  gap: 16px;
}

/* ═══════════════════════════════════════════
   WIZARD MODAL
═══════════════════════════════════════════ */
.overlay {
  position: fixed; inset: 0;
  background: rgba(0,0,0,0.7);
  backdrop-filter: blur(4px);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.wizard-modal {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 12px;
  width: 640px;
  max-width: 90vw;
  max-height: 85vh;
  overflow-y: auto;
  padding: 28px;
  position: relative;
  box-shadow: 0 20px 60px rgba(0,0,0,0.5);
}

.close-btn {
  position: absolute;
  top: 14px; right: 14px;
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text-dim);
  width: 28px; height: 28px;
  border-radius: 50%;
  cursor: pointer;
  font-size: 0.85rem;
}
.close-btn:hover { color: var(--text); }

/* ── Wizard Progress ──────────────────────── */
.wizard-progress {
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 24px;
  gap: 0;
}
```

```css
.progress-step {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
}

.progress-dot {
  width: 28px; height: 28px;
  border-radius: 50%;
  background: var(--surface2);
  border: 2px solid var(--border);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.75rem;
  font-weight: 700;
  color: var(--text-dim);
  transition: all 0.2s;
}

.progress-step.active .progress-dot {
  background: var(--accent);
  border-color: var(--accent);
  color: white;
}

.progress-step.done .progress-dot {
  background: var(--green);
  border-color: var(--green);
  color: white;
}

.progress-label {
  font-size: 0.65rem;
  color: var(--text-dim);
  text-transform: uppercase;
}

.progress-line {
  flex: 1;
  height: 2px;
  background: var(--border);
  min-width: 20px;
  margin-bottom: 14px;
}

/* ── Wizard Body ──────────────────────────────── */
.wizard-body h3 {
  font-size: 1.1rem;
  margin-bottom: 14px;
  color: var(--text);
}

.wizard-text p  { margin-bottom: 8px; line-height: 1.6; font-size: 0.9rem; }
.wizard-text ul { padding-left: 20px; }
.wizard-text li { margin-bottom: 6px; font-size: 0.9rem; }
```

```css
.wizard-text a  { color: var(--accent); }

.hint {
  color: var(--text-dim);
  font-size: 0.82rem !important;
  background: var(--surface2);
  padding: 8px 12px;
  border-radius: 6px;
  border-left: 3px solid var(--accent);
}

/* ── Wizard Status ──────────────────────────── */
.wizard-status { margin-top: 10px; font-size: 0.85rem; min-height: 24px; }
.wizard-status .ok { color: var(--green); }

/* ── Wizard Params ──────────────────────────── */
.wizard-params { display: flex; flex-direction: column; gap: 20px; }

.param-block h4 {
  font-size: 0.9rem;
  margin-bottom: 4px;
}
.param-block p {
  font-size: 0.8rem;
  color: var(--text-dim);
  margin-bottom: 8px;
}
.param-block input {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 6px 10px;
  border-radius: 6px;
  width: 120px;
  margin-top: 8px;
}

.preset-row {
  display: flex;
  gap: 8px;
  flex-wrap: wrap;
}

.preset-btn {
  background: var(--surface2);
  border: 1px solid var(--border);
  color: var(--text-dim);
  padding: 8px 14px;
  border-radius: 8px;
  cursor: pointer;
  font-size: 0.8rem;
  text-align: center;
  transition: all 0.15s;
  line-height: 1.4;
}
.preset-btn:hover  { border-color: var(--accent); color: var(--text); }
.preset-btn.active { background: var(--accent); border-color: var(--accent); color: white; }
```

```
.preset-btn small  { color: inherit; opacity: 0.8; font-size: 0.7rem; }

/* ── Wizard Done ──────────────────────────── */
.wizard-done { text-align: center; padding: 20px; }
.done-icon { font-size: 3rem; margin-bottom: 12px; }

.config-summary {
  background: var(--surface2);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 14px;
  margin: 14px 0;
  text-align: left;
}

.summary-line {
  font-size: 0.85rem;
  padding: 4px 0;
  border-bottom: 1px solid var(--border);
}
.summary-line:last-child { border: none; }
.summary-line span {
  font-family: var(--font-mono);
  color: var(--text-dim);
  font-size: 0.78rem;
  word-break: break-all;
}

/* ── Wizard Footer ──────────────────────────── */
.wizard-footer {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-top: 24px;
  padding-top: 16px;
  border-top: 1px solid var(--border);
  gap: 8px;
}
```

---

6 package.json

{} JSON

```
{
  "name": "hermes-launcher",
  "version": "2.0.0",
  "description": "Hermes LLM Launcher – Desktop GUI für llama-server",
  "main": "main.js",
  "scripts": {
    "start":  "electron .",
    "dev":    "electron . --dev",
```

```
    "build":  "electron-builder --win --x64",
    "pack":   "electron-builder --dir"
  },
  "dependencies": {
    "electron-store": "^8.2.0"
  },
  "devDependencies": {
    "electron":         "^36.0.0",
    "electron-builder": "^24.0.0"
  },
  "build": {
    "appId":   "com.hermes.launcher",
    "productName": "Hermes Launcher",
    "win": {
      "target": [{ "target": "nsis", "arch": ["x64"] }],
      "icon":   "assets/icon.ico"
    },
    "files": [
      "main.js", "preload.js",
      "renderer/**", "backend/**",
      "node_modules/**"
    ]
  }
}
```

⚡

```
{} Bash
```

```bash
# 1. Abhängigkeiten installieren
npm install

# 2. App starten
npm start

# 3. Windows-Installer bauen
npm run build
```

🗺️

📁 llama-server.exe auswählen	✅	Nativer Dialog, `.exe`-Filter, persistent
📁 GGUF-Modell auswählen	✅	`.gguf`-Filter, letzter Pfad gespeichert

🔍 Im Explorer öffnen	✅	`shell.showItemInFolder()`
📋 Live Log-Viewer	✅	Echtzeit stdout/stderr, Farb-Coding
🔍 Log-Suche	✅	Highlight, Regex-safe
🎛️ Log-Filter	✅	Nur Fehler, Auto-Scroll
💾 Log exportieren	✅	`.txt`-Download
🧑‍🔧 Setup-Assistent	✅	5-Schritte Wizard, Presets
⏱️ Uptime-Counter	✅	Läuft solange Server aktiv
💾 Persistenz	✅	`electron-store`, alle Pfade gespeichert
🔒 Sicherheit	✅	`contextIsolation`, kein `nodeIntegration`

> 📌 *Nächster Schritt: Auto-Updater, Tray-Icon, Chat-UI direkt im Launcher, oder Hugging Face Model-Downloader direkt integrieren?*